INTUTE ERP / CRM

Technical Audit: Lead-Generation Integration Readiness

Existing capabilities · Data model · APIs · Integration boundary · Gaps · Effort

Version 1.0 | 09 October 2026

Prepared for: platform research and integration-architecture evaluation

Classification: Confidential - internal use only

Document Control

Item	Detail
Document	ERP/CRM Technical Audit - Lead-Generation Integration Readiness
Version / date	1.0 - 09 October 2026
Systems inspected	CRM_BACKEND (Node/Express API) at commit e353328d; CRM (React web app); live PostgreSQL catalogue (read-only)
Method	Analysis only. No code, database or configuration changes. No customer records extracted, no secrets reproduced
Purpose	Establish what the existing ERP/CRM supports, what is missing, and how external lead sources (IndiaMART, Justdial, Google Business Profiles, directories, website, referrals, WhatsApp Business, calling systems) could be integrated with minimal new development
Audience	Technical evaluators (human or AI) researching platforms, pricing and integration architecture
Note on estimates	Effort figures in section 15 are preliminary and based on stated assumptions

Contents

Part 1: Summary and Technology . . . 3

Scope and method . . . 3

1. Executive summary . . . 3

2. Repository structure and technology stack . . . 4

3. Confirmed CRM and ERP capabilities . . . 5

Part 2: Data Model, API and Access . . . 6

4. Database entities and relationships . . . 6

5. Relevant API endpoint inventory . . . 7

6. Authentication and permissions . . . 9

Part 3: Integration Readiness and Design . . . 10

7. Existing integrations, webhooks and background jobs . . . 10

8. Lead-management capability matrix . . . 10

9. Integration-readiness assessment . . . 11

10. Recommended integration boundary . . . 12

11. Potential implementation approaches . . . 12

Part 4: Risks, Gaps and Effort . . . 13

12. Security and operational considerations . . . 13

13. Missing information and questions for the team . . . 14

14. Files and modules likely to need changes . . . 14

15. Estimated implementation complexity (PRELIMINARY) . . . 15

Appendix: Machine-Readable Summary . . . 16

16. JSON summary . . . 16

Part 1: Summary and Technology

Scope and method

This audit inspected two repositories, read-only:

- `CRM_BACKEND`: the Node/Express API. It is the main subject of this audit.
- `CRM`: the React web app.

It also read the live PostgreSQL catalogue (`information_schema`, `pg_indexes`, `pg_stat_user_tables`, `roles`, `permissions`) using SELECT statements only. No customer records were extracted. No code, database or configuration was changed. Paths are relative to `CRM_BACKEND/` unless they start with `CRM/`.

Every finding carries one of these labels:

Label	Meaning
Confirmed	Seen in the code or the live schema
Partial	Some of the capability exists
Not found	Absent from the inspected code; not proof that it is absent from the whole system
Unknown	Cannot be established from the repositories

1. Executive summary

- **A working lead pipeline exists, called "Enquiry".** The data lives in table `enquiries` and the API is `/api/enquiry` (files `routes/sales/enquiry.js`, `controllers/sales/enquiry.controller.js`, `models/sales/enquiry.js`).
  - Each enquiry stores the company, one contact (name, email, phone), city, items required, `source` (free text, default `'Website'`), `lead` (temperature: `hotlead`, `followup`, `lead`, `not_interested`, `closed`), `stage`, `status`, tags, photos, the assignee with who assigned it and when, `due_date` and `next_interaction`.
  - Each enquiry has a threaded activity log (`enquiry_activities`) with comments, @mentions and assignment entries, plus watchers and read receipts.
  - Field sales representatives (role 15) were added on 23 September 2026 and capture leads straight into this table.
- **Integration plumbing is missing.** None of the following were found in the inspected code:
  - API keys or service accounts for server-to-server calls
  - webhook receivers and outbound HTTP clients
  - fields for external platform IDs
  - idempotency handling
  - duplicate-lead detection (there is no index on phone or email)
  - CSV or Excel import
  - lead-to-customer conversion
  - structured logs for calls, messages or meetings
  - reminders for enquiry follow-ups
  - reports by source or by employee
- **Recommended boundary.** Build a connector module inside the existing Express backend: same stack, same conventions, and the cron infrastructure is already there. The connector should normalise and deduplicate incoming data, then call the existing enquiry model layer (`Enquiry.create`, `assign`, `addComment`). It should not write directly to the database. Three things must be built first: a machine-authentication mechanism, an inbound-event log table for idempotency and retries, and external-ID and normalised-phone columns with unique indexes.
- **Preliminary effort:** about 3–6 developer-weeks to make the Enquiry module integration-ready and connect 2–3 lead sources. This excludes vendor onboarding and WhatsApp or telephony platform work (section 15).

2. Repository structure and technology stack

Structure.

- The backend has three layers: `routes/<domain>/*.js` → `controllers/<domain>/*.controller.js` → `models/<domain>/*.js`. The domains are core, sales, manufacturing, invoicing, dispatch, hr, operations and service.
- `chatbot/` holds a separate rule-based Q&A module that matches phrases and uses no LLM.
- Supporting folders: `jobs/` (cron), `services/` (email, Google Drive, queries), `middleware/`, `config/`, `scripts/migrations/` and `tests/`.
- The entry point is `server.js`.

Component	Finding	Evidence / status
Backend language	JavaScript (CommonJS) on Node.js **22.17.0** in production	`Dockerfile` (`FROM node:22.17.0-alpine`). Confirmed
Backend framework	Express **^4.19.2**	`package.json`, `server.js`. Confirmed
Frontend	React **^19.0.0**, Vite **^6.2.0**, Redux Toolkit ^2.11, React Router ^7.2, Tailwind 3, socket.io-client, `xlsx`	`CRM/package.json`, `CRM/src/routeConfig.jsx`. Confirmed (separate repository)
Database	PostgreSQL on AWS RDS over TLS; 61 public tables, 474 columns	`config/db.js` and the live catalogue. **Server version not queried (Unknown)**
ORM / data access	**No ORM.** Raw parameterised SQL through the `pg` ^8.13 `Pool` (max 20 connections). Transactions use `pool.connect()` + `BEGIN/COMMIT/ROLLBACK`	`config/db.js`, `models/**`. Confirmed
Authentication	Email + bcrypt password login issues a JWT signed with `JWT_SECRET`. Lifetime is 1 hour, or 12 hours when the client sends an `X-App-Version` header. Accepted only as a Bearer header, never from a cookie	`controllers/auth.controller.js` (`login`), `middleware/auth.js` (`authenticateToken`). Confirmed
Authorisation	`roles` table (14 roles) plus `permissions(role_id, module, can_read, can_write, can_delete)`, checked by `checkPermission(module, action)`. Some routes add hard-coded role allow-lists or "assigned to me" row rules	`middleware/auth.js`, `routes/operations/activities.js` (`ALLOWED_ROLES`), `models/sales/enquiry.js`. Confirmed
API style	REST/JSON, with Socket.IO push to the browser	`server.js`. Confirmed
Background processing	`node-cron` jobs run inside the API process on IST. **No queue and no workers**	`jobs/daily-due-reminders.js`, `jobs/daily-task-summaries.js`, `jobs/attendanceSummary.js`. Confirmed
Deployment	Docker image `intuteai/erp-backend:latest`, built by a **manually triggered** GitHub Action and deployed over SSH to EC2 with `docker-compose`. nginx sits in front (`trust proxy = 1`)	`.github/workflows/backend-erp.yml`, `Dockerfile`. **nginx and EC2 configuration not in the repository (Unknown)**
Configuration	`dotenv` `.env` file (DB_*, JWT_SECRET, REDIS_URL, EMAIL_*, FRONTEND_URL, GOOGLE_*, MANAGER_DAILY_REPORT_EMAIL, DEFAULT_SALES_USER_ID). The Google service-account key lives under `secrets/`	`config/*.js`, `README.md`. Confirmed (values not inspected)
Testing	Jest 29 + Supertest. Coverage is concentrated on other modules and on auth. Enquiry tests exist (`tests/enquiry_controller.test.js`, `tests/enquiry_model.test.js`); there are **none for customers, orders or activities**	`tests/`. Confirmed
Monitoring	Winston logs (`logs/error.log`, `logs/combined.log`, console). The only health check is `GET /`, which returns a welcome JSON. **No error tracker or metrics found**	`utils/logger.js`, `server.js`. Confirmed
Cache	Redis (node-redis 4). List caches expire after 300 s	`config/redis.js`. Confirmed
Other integrations	Gmail SMTP through Nodemailer; Google Drive API for photo and PDF storage	`services/email.js`, `services/googleDrive.js`. Confirmed

3. Confirmed CRM and ERP capabilities

Confirmed modules:

- **Sales:** enquiries (the lead pipeline), enquiry technical requirements, customers, price list
- **Orders and stock:** orders with inventory holds, stock deducted at shipping, finished-goods inventory, raw-material stock
- **Documents:** quotation, proforma and purchase-order PDF generators. These are **stateless**: they return a PDF and save no record.
- **Finance and dispatch:** customer and purchase invoices, dispatch tracking, delivery challans
- **Manufacturing:** BOM, parts, work orders, processes and testing stages, IPT kits
- **Service:** service and repair
- **HR:** attendance, payslips, employee details
- **Internal:** tasks ("activities"), customer queries (tickets), problems

Sales-related front-end roles (`CRM/src/routeConfig.jsx`):

Role	Pages	Visibility
admin	`/enquiries`, `/customer-list`, orders, invoices	All records
sales	`/sales/enquiries`, `/sales/quotations`	All enquiries
design	`/design/enquiries`	Only enquiries assigned to them
representative	`/representative/enquiries`	Only their own leads

Part 2: Data Model, API and Access

4. Database entities and relationships

All entries below come from the live catalogue. Row counts are approximate and only show scale.

Table	Important fields	PK	Foreign keys	Unique	Search / dedup indexes	Timestamps
`users` (~100)	name, email, password_hash, role_id	user_id	none (no FK from role_id to roles)	email	`unique_email`	created_at
`roles` (14) / `permissions` (95)	role_name / module + can_read, can_write, can_delete	role_id / permission_id	none (no FK from permissions.role_id)	none	none	none
`employee_details` (~39)	employee_id, phone_number, date_of_joining	employee_id	user_id → users	none	none	created, updated
`customers` (~35)	contact_person, city, phone, gst, shipping and billing address	customer_id	user_id → users	user_id	none on phone or gst	created_at
`enquiries` (~17)	enquiry_id (business ID), company_name, contact_person, mail_id, phone_no, city, items_required, application, **source** (varchar; no FK, no lookup table), **lead**, **stage**, **status**, tags text[], photos text[], assigned_to / assigned_by / assigned_at, due_date, last_discussion (**a timestamp**), next_interaction, created_by	sr_no	assigned_to, assigned_by, created_by → users	enquiry_id	company_name, status, stage, assigned_to, created_by, due_date, tags, lead. **None on phone_no, mail_id or source**	created, updated
`enquiry_activities` (~77)	activity_type (comment, assignment, system), message, mentions int[], expected_by, is_internal	activity_id	enquiry_id → enquiries.enquiry_id; user_id → users	none	type, enquiry, mentions, created	created_at
`enquiry_watchers`, `enquiry_read_receipts`	(enquiry_id, user_id) / (activity_id, user_id)	composite	yes	none	none	watched_at / read_at
`comment_templates` (5)	title, content, is_global	id	created_by → users	none	none	created_at
`enquiry_requirements` / `enquiry_requirement_motors`	id, enquiry_id (**bigint with no FK**, while enquiries.enquiry_id is a varchar), title, description / motor specification	id	motors.requirement_id → requirements	none	motor_type, controller	created, updated
`activities` (~50) + `activity_assignees`	summary, status, due_date, priority, comments, org (enum)	id	created_by → users; assignees → users and activities	none	status, priority, assignee	created, updated
`queries` / `query_responses`	query_text, query_status / response	query_id / response_id	none declared	none	none	yes
`orders` / `order_items`	user_id (the customer's **user** account), status (enum), payment_status, status_reason, target_delivery_date / product_id, quantity, price	order_id / order_item_id	orders.user_id: **no FK**; order_items → orders, inventory	none	none	created_at
`customer_invoices`	invoice_number, total_value, link_pdf	invoice_id	order_id → orders; customer_id → customers	none	none	created, updated
`inventory`, `price_list`, `raw_materials`	product master, prices, stock	various	price_list.product_id → inventory	part_number, product_code	n/a	n/a

How prospects are represented today.

- There is **no separate lead, contact, organisation, opportunity or deal entity**. A prospect is one row in `enquiries`, which holds one company and one contact. Qualification, temperature, pipeline stage and the opportunity all live on that row.

- A customer is a `users` row with role 2 (`customer`, which can log in to the portal) plus a `customers` row (`models/core/customer.js` → `User.create`).
- **No foreign key or code links `enquiries` to `customers` or `orders`.**

Gaps that matter for lead generation:

1. There is no column for an external platform's record ID or its raw payload, and `source` is free text with no constraint.
2. Phone numbers and emails are not normalised or indexed, so duplicate detection would need a full scan with LIKE.
3. An enquiry can hold only one contact.
4. There is no interactions table with a type (call, WhatsApp, meeting), direction, duration or outcome; only free-text comments.
5. `last_discussion` records *when* the last conversation happened, not what was said.
6. There is no structured lost reason. `closed_lost` is a stage, and a reason can only be written as a comment.
7. Leads cannot be converted to customers, so there is no way to attribute revenue back to a lead source.
8. The model code and the live table disagree. In the live DB, `enquiry_requirements` has 6 columns, but `models/sales/enquiryRequirements.js` also selects `requirement_type`, `assignee_id`, `attachments`, `metadata` and others. Those endpoints are therefore likely to fail in production. **Needs verification.**
9. The enquiry ID generator, `get_next_enquiry_id()`, is a database function that is not defined in the repository.

5. Relevant API endpoint inventory

All of these endpoints require `Authorization: Bearer <user JWT>`. "Perm" means a `checkPermission(module, action)` check.

Capability	Method	Route	Purpose	Auth / permission	Request → response	Evidence
Create lead	POST	`/api/enquiry`	Create an enquiry	Enquiries:write	`company_name` (required), contact_person, mail_id, phone_no, items_required, source (default 'Website'), application, lead/priority, tags[], status, assigned_to, due_date, city, last_discussion, next_interaction, optional enquiry_id → enquiry row. Only company_name is validated. A duplicate enquiry_id returns 400 `DUPLICATE_ENQUIRY_ID`	`create`, `Enquiry.create`
List / search leads	GET	`/api/enquiry`	Paginated list	Enquiries:read; design and representative roles see only their own	`limit`, `offset`, `cursor`, `search` (LOWER LIKE over id, company, contact, mail, phone, items, status, application). **No server-side filter for stage, source, assignee or date**	`getAll`
Get lead	GET	`/api/enquiry/:id`	Enquiry with its activities	Enquiries:read + ownership	Enquiry + activities	`getOne`
Update lead	PUT	`/api/enquiry/:id`	Update fields	Enquiries:write + ownership	Same fields as create. **`last_discussion` and `next_interaction` are cleared if omitted**; other fields use COALESCE and keep their values	`Enquiry.update`
Assign owner	POST	`/api/enquiry/:id/assign`	Reassign	Enquiries:write	`assigned_to` (required), due_date, note. Sets stage `in_discussion` and status `In Progress`, and logs an assignment activity	`Enquiry.assign`

Capability	Method	Route	Purpose	Auth / permission	Request → response	Evidence
Change stage	PATCH	`/api/enquiry/:id/stage`	Move through the pipeline	Enquiries:write	`stage` must be one of closed_won, closed_lost, regret, in_discussion, design_review, admin_review; optional note. Status is derived from the stage	`changeStage`
Record activity	POST	`/api/enquiry/:id/comment`	Comment, @mention, expected-by date	Enquiries:write	message (required), mentions[], expected_by, is_internal	`addComment`
Hand back to Sales	POST	`/api/enquiry/:id/mark-done`	Design returns the enquiry to the default sales user	Enquiries:write; design role only	Uses `DEFAULT_SALES_USER_ID`	`markDone`
Follow / read receipts	POST, DELETE	`/api/enquiry/:id/follow`, `/:enquiryId/activity/:activityId/read`	Follow; mark as read	Enquiries:read	n/a	controller
Photos	POST, DELETE	`/api/enquiry/:id/photo`	Upload or remove a photo	Enquiries:write	multipart, 10 MB limit; stored on Google Drive with a public link	`uploadPhoto`
Assignable representatives	GET	`/api/enquiry/representatives`	Users with the representative role	Enquiries:read	n/a	`getRepresentatives`
Comment templates / cache refresh	GET / POST	`/api/enquiry/templates`, `/api/enquiry/refresh`	Canned comments; clear the cache	Enquiries:read	n/a	controller
Create customer	POST	`/api/customers`	Creates a **login user (role 2)** plus a customer row	Customers:write	name, email, contact_person, city, phone (10 digits), optional gst, shipping and billing address	`Customer.create`
Search / list customers	GET	`/api/customers/search?q=`, `/api/customers`	ILIKE on name, limit 20 / paginated list	Login only for search / Customers:read for the list	n/a	customers controller
Update customer	n/a	not exposed	`Customer.update` exists in the model but **no route uses it**	n/a	n/a	`models/core/customer.js`
Follow-up task	POST	`/api/activities`	Internal task with assignees and a due date	**role_id 9, 10, 11 or 12 only (HR and employee roles), not sales**	n/a	`routes/operations/activities.js`
Orders	GET, POST, PUT	`/api/orders`, `/api/orders/export` (CSV)	Sales orders; the customer is a `user_id`	Orders:*	n/a	orders controller
Quotation / proforma	POST	`/api/quotation/generate`, `/api/proforma/generate`	PDF only, **not persisted**	Login only	n/a	`routes/sales/*.js`
Invoices	CRUD	`/api/customer-invoices`	Invoice register	CustomerInvoices:*	n/a	invoicing controller
Reports	GET	`/api/reports/order-summary`, `/api/reports/inventory-status`, `/api/dashboard/admin-stats`	Order totals, low stock, counts	Reports / dashboard	**No lead, source or employee funnel reports**	reports and dashboard controllers
Duplicate check	n/a	**Not found**	n/a	n/a	n/a	n/a
Integration / sync trigger	n/a	**Not found**	n/a	n/a	n/a	n/a
Opportunity CRUD	n/a	**Not found** as a separate entity; the enquiry stage acts as the opportunity	n/a	n/a	n/a	n/a

Live `Enquiries` permissions in the `permissions` table:

Role	Enquiries access
admin	read, write, delete
sales	read, write
design	read, write (only their own rows, enforced in code)
representative	read, write (only their own rows, enforced in code)

Role	Enquiries access
all other roles	none

6. Authentication and permissions

- Login is email + password and returns a JWT. The token is accepted only in the `Authorization` header; the README's claim that cookies also work is not supported by the code.
- `authenticateToken` re-reads the user and role from the database on every request, so access changes take effect immediately.
- **There is no machine-to-machine authentication**: no API keys, no client credentials, no OAuth flow and no per-integration scopes.
  - A connector built inside the backend could call model functions directly.
  - An external tool would need a dedicated user account that logs in and renews its 1-hour token. A 12-hour token is possible through the self-declared `X-App-Version` header, but neither option is a proper machine identity.
- **Row rules:** `models/sales/enquiry.js` (`assertMutationAllowed`) limits design and representative roles to rows assigned to them. Sales and admin see everything.
- **Fails closed:** a role with no `permissions` row for a module is denied. An integration module would need new rows.
- **Logout** is stateless. Tokens are never revoked.

Part 3: Integration Readiness and Design

7. Existing integrations, webhooks and background jobs

Item	Status	Evidence
Outbound HTTP clients (axios, fetch) in the backend	**Not found**	Search across all backend `.js` files
Webhook endpoints or signature verification	**Not found**	Same search
Google Drive: file upload and trash (service account)	Confirmed	`services/googleDrive.js`
Gmail SMTP email	Confirmed	`services/email.js`, `utils/emailTemplates.js`
Socket.IO push to the browser	Confirmed for broadcasts. Notifications to individual users are sent to `user_<id>` rooms, but **no code adds sockets to those rooms**, so they probably never arrive	`server.js`, `models/sales/enquiry.js`
Cron jobs	Confirmed, all email-only, IST: reminders for activities due tomorrow (11:00); task summaries (09:00 and 18:30); attendance summaries (10:00 and 19:30)	`jobs/*.js`
Enquiry follow-up / overdue reminders	**Partial (dead code):** `Enquiry.checkOverdue(io)` exists but is never scheduled, so `next_interaction` and `due_date` trigger nothing	`models/sales/enquiry.js`
Queue, retry, dead-letter handling	**Not found**	n/a
CSV / Excel import	**Not found.** Exports exist: server CSV for orders, and client-side xlsx for inventory, stock, attendance and orders. **There is no enquiry import or export**	`CRM/src/components/*`
Idempotency keys / event deduplication	**Not found**	n/a
WhatsApp, telephony or SMS providers	**Not found**	n/a

8. Lead-management capability matrix

Capability	Status	Evidence	How it works	Gaps
Lead records	Confirmed	`enquiries`, `Enquiry.create`	One row per lead	Company and contact are mixed into one row
Contact records	Partial	enquiries.contact_person / mail_id / phone_no; customers.contact_person / phone	One contact per record	No contacts table
Business profile	Partial	company_name, city, application, items_required, tags	Free text	No GST, address, category, location, rating or website on leads
Create / update	Confirmed	POST and PUT `/api/enquiry`	n/a	PUT clears follow-up fields that are omitted; only light validation
Status / qualification	Confirmed	`status`, `lead`, `stage`	Three separate fields	Transition order not enforced; no qualification criteria
Source attribution	Partial	`enquiries.source`	Free text, default 'Website'	No lookup table, sub-source, campaign or external ID
Ownership / assignment	Confirmed	assigned_to / by / at, `/assign`, activity log	Manual	No round-robin or rule-based assignment
Duplicate detection	Not found	n/a	n/a	No normalisation, index or check endpoint
Search / filter	Partial	`search` parameter	LIKE over several fields	No stage, source, owner or date filters on the server

Capability	Status	Evidence	How it works	Gaps
Import / export (enquiries)	Not found	n/a	n/a	n/a
Customer conversion	Not found	n/a	Customers are created separately as portal users	No link from enquiry to customer
Pipeline	Confirmed (simple)	`stage` values	new → in_discussion → design_review or admin_review → closed_won, closed_lost or regret	No deal value, probability or expected close date
Opportunities / deals	Not found as an entity	n/a	The enquiry acts as the opportunity	No amount or product lines on enquiries
Activities / notes	Confirmed	`enquiry_activities`, `/comment`	Threaded, @mentions, read receipts	Only three activity types (comment, assignment, system)
Call logs	Not found (structured)	n/a	Comments plus the last_discussion timestamp	No direction, duration, recording or outcome
Email / message history	Not found	n/a	n/a	n/a
Tasks / reminders	Partial	`activities` + cron email	Restricted to roles 9–12	Not linked to enquiries; sales cannot use it
Meetings	Not found	n/a	n/a	n/a
Follow-up dates	Partial	next_interaction, due_date, expected_by	Stored only	No reminder job
Quotations	Partial	`/api/quotation/generate`	Generates a PDF	Not stored; not linked to an enquiry
Orders / invoices	Confirmed	`/api/orders`, `/api/customer-invoices`	n/a	Not linked to the enquiry or its source
Lost-deal reasons	Partial	closed_lost stage + a comment note	n/a	Not structured
Lead dashboards / reports	Not found in backend	n/a	The front end may calculate counts in the browser (Unknown)	No source, employee or conversion metrics

9. Integration-readiness assessment

Capability	Classification
External service creates leads through a supported endpoint	**Requires a new integration component.** `POST /api/enquiry` exists but needs a user JWT
Search for an existing lead before creating one	**Requires a small code change:** exact match on normalised phone and email, plus indexes
Update records safely	**Requires a small code change:** PUT clears omitted follow-up fields, and there is no optimistic locking
Store the lead source	**Available with configuration** (free-text `source`); a lookup table needs a small change
Record external platform IDs	**Requires a small code change:** new columns with a unique index
Assign leads to employees	**Ready to use:** `/assign`, `assigned_to`
Create activities	**Ready to use** for notes (`/comment`). **Requires a new component** for typed call, message and meeting logs
Follow-up tasks and reminders	**Requires a small code change:** schedule a reminder job on `next_interaction` / `due_date`, or open `activities` to sales roles
Server-to-server authentication	**Requires a new integration component:** API-key or service-account auth
Respect role permissions	**Available with configuration:** new `permissions` rows
Receive and process webhooks	**Requires a new integration component**
Scheduled imports	**Available with configuration:** the `node-cron` pattern exists, but each job is new code

Capability	Classification
Retry failures safely	**Requires a new integration component:** there is no queue or event log
Handle duplicate events	**Requires a new integration component:** an idempotency table
Rate limits and pagination	Incoming requests are limited globally to 5,000 per 10 minutes per user or IP (`middleware/rateLimit.js`). Vendor-side limits are **not established**
Monitor integration failures	**Requires a new integration component.** Today there are only Winston log files; email alerts could reuse `services/email.js`

10. Recommended integration boundary

```
External source (IndiaMART pull API / Justdial / webhook / CSV / WhatsApp BSP / IVR)
   -> Connector inside CRM_BACKEND (new services/integrations/<source>.js + routes/integrations/*)
   -> inbound_events log (raw payload, source, external_id, received_at, status, attempts, error)
   -> Normalise + validate (phone to E.164, lower-case email, required fields)
   -> Deduplicate (unique external_id; then normalised phone/email match on enquiries)
   -> Existing model layer: Enquiry.create / update / assign / addComment
   -> enquiries + enquiry_activities (system comment "Imported from <source> #<id>")
   -> Socket.IO enquiryUpdate (already emitted by the model) + optional email alert
```

- **Why go through the model layer.** The existing model functions already enforce the business rules, write the activity log, send Socket.IO events and clear the cache. A direct database write from an automation tool (Zapier, n8n, Make) would skip all of that: no activity trail, a stale Redis cache, no events, and no handling of the `enquiry_id` generator function.
- **Webhooks.** Webhook handlers should reply 200 straight after saving the raw event, then process it in the background. With no queue today, that means `setImmediate` or a cron sweep.
- **CRM-to-ERP consistency.** Linking a won enquiry to its customer and orders needs an explicit `customer_id` or `user_id` link on the enquiry, set during conversion.
- **Retention and audit.** Keep raw payloads in the event table under a retention policy. Field-level edit history does not exist today: PUT updates are not logged.
- **Concurrent updates.** Only `markDone` locks the row (`SELECT … FOR UPDATE`). Everywhere else the last write wins. That matters when a webhook and a user edit the same lead at the same time.

11. Potential implementation approaches

Approach	Description	Fit
A. Connector module inside the backend (recommended)	New `services/integrations/`, `routes/integrations/` (webhooks + admin replay), `jobs/<source>Pull.js`, an `inbound_events` table and API-key middleware	Same stack and conventions; cron is already there; reuses the model layer. Downside: cron runs inside the API process, so long imports share CPU and memory with users
B. Separate integration microservice	A small Node worker with a queue (Redis already exists) calling a new authenticated `/api/integrations/leads` endpoint	Better isolation and retries. Still needs machine auth, plus its own deployment pipeline
C. iPaaS (n8n, Make, Zapier) calling the backend API	The vendor tool polls sources and POSTs to a new API-key endpoint	Fastest way to connect many directories. Still needs machine auth and deduplication on the server; must not write to the DB directly
D. CSV / Excel import screen	Upload → preview → deduplicate → bulk create	Covers Justdial and Google Maps exports and event lists. The front end already includes `xlsx`

Part 4: Risks, Gaps and Effort

12. Security and operational considerations

These findings are sensitive. No secret values are reproduced here.

Secrets and credentials

- The `Dockerfile` copies `.env` into the image (`COPY .env .env`). The image is pushed to Docker Hub as `intuteai/erp-backend:latest`, and whether that repository is private is Unknown.
- When a customer user is created without a password, `User.create` gives it a hard-coded default password hash. Customers are users who can log in.

Access control

- Several routes only check that the caller is logged in, with no permission check: the quotation, proforma and purchase-order PDFs, purchase invoices, BOM, part drawings, payslips, the chatbot, and `GET /api/customers/search`. New integration endpoints should not copy this pattern.

Personal data

- Enquiry and service photos are stored on Google Drive with public `anyone:reader` links.
- Lead data has no consent, retention or deletion workflow. Deleting an enquiry permanently deletes it along with its activity history.
- The data includes phone numbers, emails and photos of businesses and people.

Input validation

- Enquiries validate only `company_name`.
- Customers have stricter checks (a 10-digit phone and the GST format).
- HTML sanitisation (`sanitize-html`) is used in some modules, but not in enquiries.

Rate limiting

- Global limit: 5,000 requests per 10 minutes, keyed by user or IP. The IP is correct behind nginx because of `trust proxy 1`.
- The inventory routes have a tighter limit: 100 requests per 15 minutes.
- There is no rate limit per API key.

Audit trail

- `enquiry_activities` records assignments, stage changes and comments.
- Field edits are not recorded, and there is no general audit log.

Monitoring

- The only health endpoint is `GET /`.
- No APM tool, error tracker or alerting was found.
- Logs are written inside the container. Whether they are shipped anywhere is Unknown.

Reliability gaps

- Some cache invalidations do nothing, because `redis.del` is called with a literal wildcard key.
- Stock adjustments are not atomic.
- A shared `io.user` singleton in some routers can mix up concurrent requests.

13. Missing information and questions for the team

Unknown	Evidence that would resolve it
PostgreSQL server version and the DB user's privileges (can it run DDL?)	`SELECT version()` and the role grants on RDS
nginx, TLS, firewall, and whether webhook URLs can be reached from outside	nginx config and EC2 security groups
Whether the Docker Hub image is private, and how production secrets are stored	Docker Hub settings; the server's `docker-compose.yml`
Database objects not in the repo: `get_next_enquiry_id()` and the order/dispatch sync triggers	Definitions in `pg_proc` and `pg_trigger`
Whether the enquiry-requirements endpoints work despite the schema mismatch	Call the endpoint, or check the logs
Lead volumes per source per day, and expected peak webhook bursts	Business estimates; vendor plans
Which sources will be subscribed to, and what each offers (API, webhook, export) and at what contract limits	IndiaMART Lead Manager API terms, Justdial, Google Business Profile API access, WhatsApp BSP choice
Calling setup (IVR or cloud telephony?) and whether recordings must be stored	Operations team
The value of `DEFAULT_SALES_USER_ID`, and who should receive unassigned leads	`.env` (value not inspected); the sales manager
How employees use enquiries today: is `source` filled in consistently, and are leads also kept in spreadsheets?	Interviews; an aggregate count of enquiries by source
Whether front-end dashboards calculate lead metrics in the browser	Review `CRM/src/components/dashboards/RoleDashboard.jsx` and the sales pages
Data-protection obligations (DPDP Act 2023) for scraped or purchased lead data	Legal / compliance
Where logs are shipped, and the RDS backup and retention policy	AWS console

14. Files and modules likely to need changes

File / module	Change
`models/sales/enquiry.js`, `controllers/sales/enquiry.controller.js`, `routes/sales/enquiry.js`	Duplicate lookup, upsert by external ID, filters (source, stage, owner, date), safe partial update, conversion
`middleware/auth.js` or a new `middleware/apiKey.js`	Machine authentication with scopes
New `services/integrations/*`, `routes/integrations/*`, `controllers/integrations/*`	Webhook receivers, pull clients, normaliser
New `jobs/*Pull.js`, `jobs/enquiryReminders.js`	Scheduled imports; follow-up reminders (or wire up `Enquiry.checkOverdue`)
New `scripts/migrations/<date>-*.js`	`external_source` and `external_id`, normalised phone and email with indexes, `inbound_events`, interaction-type fields, a customer link, a lost-reason field
`server.js`	Mount the new routers and load the new jobs
`models/core/customer.js`, `controllers/core/customers.controller.js`	Convert an enquiry into a customer; fix the non-atomic user + customer creation
`permissions` table rows	New modules (for example `Integrations`, `LeadImport`)
`services/email.js`, `utils/emailTemplates.js`	Failure alerts and reminder emails
`CRM/src/components/{sales,admin,representative}/*EnquiryPage.jsx`, `CRM/src/config/roleNav.js`	Source filter, duplicate warnings, import screen, call and meeting log screens
`tests/`	New suites for deduplication, idempotency and webhook auth. `*.test.js` is gitignored, so new tests need `git add -f`

15. Estimated implementation complexity (PRELIMINARY)

These figures assume one experienced developer who knows the codebase. They exclude vendor onboarding, legal review and setting up WhatsApp or telephony providers.

Work package	Estimate
Machine authentication (API keys with scopes) + permission rows	2–4 days
Schema additions (external ID, normalised contact fields, indexes, `inbound_events`, customer link, lost reason)	2–3 days
Deduplication and idempotent upsert service on top of `Enquiry`	3–5 days
One webhook connector (for example website form or WhatsApp BSP)	2–4 days each
One scheduled pull connector (for example the IndiaMART pull API)	3–5 days each
CSV / Excel import with preview and deduplication (backend + screen)	4–6 days
Enquiry follow-up reminders + typed interactions (call, meeting, message)	4–7 days
Funnel reports by source and employee (API + dashboard)	4–6 days
Tests, monitoring and alerting	3–5 days
Indicative total for the core work plus 2–3 sources	**About 3–6 developer-weeks**

Appendix: Machine-Readable Summary

16. JSON summary

This summary lists only verified facts and explicit unknowns. It contains no credentials or customer data.


```
{
  "technology_stack": {
    "backend": {"language": "JavaScript (Node.js)", "runtime_version": "22.17.0 (Dockerfile)",
                "framework": "Express ^4.19.2", "api_style": "REST/JSON", "realtime": "Socket.IO ^4.8"},
    "frontend": {"framework": "React ^19.0.0", "build": "Vite ^6.2.0", "state": "Redux Toolkit ^2.11",
                 "repo": "CRM (separate)"},
    "cache": "Redis (node-redis ^4.7)",
    "email": "Nodemailer via Gmail SMTP",
    "file_storage": "Google Drive API (service account)",
    "testing": "Jest 29 + Supertest; enquiry tests present; none for customers, orders, activities",
    "monitoring": "Winston file/console logs; GET / only; no APM or error tracker found",
    "deployment": "Docker image on AWS EC2 via docker-compose; manual GitHub Actions trigger; nginx (conf
    ig not in repo)"
  },
  "database": {
    "engine": "PostgreSQL (AWS RDS)",
    "server_version": "unknown",
    "access_layer": "raw parameterised SQL via pg Pool; no ORM",
    "migrations": "hand-run Node scripts in scripts/migrations",
    "tables_public": 61,
    "db_side_objects_not_in_repo": ["get_next_enquiry_id()", "order/dispatch status sync triggers"]
  },
  "authentication": {
    "method": "email+password login issuing JWT (Bearer header only)",
    "token_ttl": {"default": "1h", "with_X-App-Version_header": "12h"},
    "machine_to_machine": "not found (no API keys, OAuth client credentials or service accounts)",
    "authorisation": "permissions table via checkPermission; row-level own-records rule for design and re
    presentative roles on enquiries",
    "token_revocation": "not found"
  },
  "api_capabilities": {
    "create_lead": "POST /api/enquiry (Enquiries:write)",
    "list_search_leads": "GET /api/enquiry?limit&offset&cursor&search (text LIKE; no structured filters)"
    ,
    "get_lead": "GET /api/enquiry/:id",
    "update_lead": "PUT /api/enquiry/:id (overwrites last_discussion/next_interaction when omitted)",
    "assign_lead": "POST /api/enquiry/:id/assign",
    "change_stage": "PATCH /api/enquiry/:id/stage",
    "add_activity": "POST /api/enquiry/:id/comment",
    "create_customer": "POST /api/customers (creates login user role 2 + customers row)",
    "update_customer": "model method only, no route",
    "duplicate_check": "not found",
    "lead_to_customer_conversion": "not found",
    "integration_trigger": "not found",
    "lead_reports": "not found in backend"
  },
  "lead_management": {
    "entity": "enquiries",
    "fields": ["enquiry_id", "company_name", "contact_person", "mail_id", "phone_no", "city",
               "items_required", "application", "source", "lead", "stage", "status", "tags", "photos",
               "assigned_to", "assigned_by", "assigned_at", "due_date", "last_discussion (timestamp)",
               "next_interaction", "created_by", "created_at", "updated_at"],
    "source_attribution": "free-text varchar, default 'Website', no lookup table, not indexed",
    "external_reference_id": "not found",
    "lead_temperature_values": ["hotlead", "followup", "lead", "not_interested", "closed"],
    "stage_values": ["new", "in_discussion", "design_review", "admin_review", "closed_won", "closed_lost"
    , "regret"],
    "status_values": ["Pending", "In Progress", "Closed", "Cancelled"],
    "dedup": "not found; no index on phone_no or mail_id",
    "contacts_per_lead": 1,
```

```
    "import": "not found",
    "export": "not found for enquiries",
    "roles_with_access": {"admin": "RWD", "sales": "RW", "design": "RW (own only)", "representative": "RW
     (own only)"}
  },
  "activity_management": {
    "lead_activity_log": "enquiry_activities (comment, assignment, system; mentions; expected_by; read re
    ceipts)",
    "typed_calls_messages_meetings": "not found",
    "tasks": "activities + activity_assignees, restricted to role_id 9,10,11,12; not linked to enquiries"
    ,
    "reminders": "cron email for activities due tomorrow; Enquiry.checkOverdue exists but is never schedu
    led"
  },
  "webhook_support": {"inbound": "not found", "outbound": "not found", "signature_verification": "not fou
    nd"},
  "background_jobs": {
    "scheduler": "node-cron inside the API process, Asia/Kolkata",
    "jobs": ["daily-due-reminders 11:00", "daily-task-summaries 09:00 and 18:30",
             "attendance summaries 10:00 and 19:30"],
    "queue_or_workers": "not found",
    "retry_mechanism": "not found",
    "idempotency": "not found"
  },
  "integration_patterns": {
    "service_location": "services/ (email.js, googleDrive.js)",
    "http_client": "none in backend",
    "transactions": "pool.connect + BEGIN/COMMIT/ROLLBACK in models",
    "validation": "express-validator in some routes; inline checks in controllers",
    "errors": "JSON {error, code}; global errorHandler",
    "logging": "utils/logger.js (Winston)",
    "realtime_events": "req.io.emit (e.g. enquiryUpdate, enquiryActivity)",
    "recommended_boundary": "in-backend connector module calling the Enquiry model layer; no direct DB wr
    ites from external tools"
  },
  "relevant_entities": ["users", "roles", "permissions", "employee_details", "customers", "enquiries",
    "enquiry_activities", "enquiry_watchers", "enquiry_read_receipts", "comment_templates",
    "enquiry_requirements", "enquiry_requirement_motors", "activities", "activity_assignees",
    "queries", "query_responses", "orders", "order_items", "customer_invoices", "price_list", "inventory"
    ],
  "relevant_endpoints": [
    "POST /api/auth/login",
    "GET|POST /api/enquiry", "GET|PUT|DELETE /api/enquiry/:id",
    "POST /api/enquiry/:id/assign", "PATCH /api/enquiry/:id/stage", "POST /api/enquiry/:id/comment",
    "POST /api/enquiry/:id/mark-done", "GET /api/enquiry/representatives", "GET /api/enquiry/templates",
    "POST|DELETE /api/enquiry/:id/photo",
    "GET|POST /api/customers", "GET /api/customers/search",
    "GET|POST|PUT|DELETE /api/activities",
    "GET|POST /api/orders", "GET /api/orders/export",
    "POST /api/quotation/generate", "POST /api/proforma/generate",
    "GET /api/reports/order-summary", "GET /api/dashboard/admin-stats"
  ],
  "security_considerations": [
    "No machine-to-machine authentication mechanism",
    ".env copied into the Docker image (registry visibility unknown)",
    "Hard-coded default password hash used when creating customer users without a password",
    "Several routes require login only, with no permission check",
    "Enquiry and service photos stored as public Google Drive links",
    "No retention, consent or deletion workflow for personal data in leads",
    "Enquiry field edits are not audited; only assignment, stage and comments are logged",
    "Per-user socket notifications are emitted to rooms that no code joins"
  ],
  "unknowns": [
    "PostgreSQL server version and DB role privileges",
    "nginx/TLS/firewall config and public reachability for webhooks",
    "Docker Hub image visibility and production secret storage",
    "Definitions of get_next_enquiry_id() and the order/dispatch triggers",
    "Whether enquiry_requirements endpoints work given the live-schema mismatch",
    "Lead volumes per source and vendor API/webhook availability and limits",
    "Telephony/WhatsApp providers in use",
    "Whether front-end dashboards compute lead metrics client-side",
    "Current consistency of the source field in existing enquiries",
```

```
    "Data-protection obligations for third-party lead data"
  ]
}
```